LEARN BEING FORWARD

Programming — JavaScript Programming

Functions, closures & scope

COMPLETE LEARNING GUIDE

What You'll Learn

Functions are the central unit of reuse and abstraction in JavaScript, but understanding them deeply means going beyond 'how to write one' into how scope is actually resolved, how a function 'remembers' the variables around it after its outer context has finished (closures), and how the notoriously confusing this keyword gets its value in four genuinely different ways depending on how a function is called. This sub-module builds that depth: the practical differences between declarations, expressions, and arrow functions; lexical scoping and the scope chain; closures as the mechanism behind private state and memoization; this-binding rules and the call/apply/bind trio that control them explicitly; higher-order functions like map/filter/reduce; and the historical IIFE/module pattern that closures made possible before ES modules existed.

1. Function Declarations, Expressions, and Arrow Functions

A function declaration (function greet() {}) is fully hoisted — both its name and its body are available anywhere in the enclosing scope, even before the line it's written on. A function expression (const greet = function() {}) is only partially hoisted: the variable declaration is hoisted, but the function assignment is not, so calling it before that line throws. Arrow functions (const greet = () => {}) are always expressions, are more concise for short bodies, have no arguments object of their own, cannot be used as constructors with new, and — most significantly — do not have their own this, instead inheriting it lexically from whatever scope they were defined in.

JAVASCRIPT

```
sayHi(); // works — function declarations are fully hoisted
function sayHi() { console.log("Hi!"); }

// sayBye(); // TypeError: sayBye is not a function (hoisted as undefined, not yet assigned)
const sayBye = function () { console.log("Bye!"); };
```

2. Lexical Scope and the Scope Chain

JavaScript resolves a variable reference by lexical scope: the engine looks for the variable in the current function's scope, and if not found there, walks outward through each enclosing function's scope, and finally the global scope — this ordered sequence of nested scopes is the scope chain. 'Lexical' means this nesting is determined by where functions are physically written in the source code, not by which function happens to call which at runtime, which is why an inner function always has access to its outer function's variables regardless of when

or from where it's eventually invoked.

```javascript
function outer() {
  const outerVar = "I'm from outer";
  function inner() {
    console.log(outerVar); // found by walking up the scope chain
  }
  inner();
}
outer(); // "I'm from outer"
```

3. Closures in Depth: Private State and Factories

A closure is what results when an inner function is returned (or otherwise escapes) from its outer function while retaining a live reference to that outer function's variables — those variables continue to exist in memory for as long as the closure itself exists, even though the outer function has already finished running. This is the mechanism behind private state in JavaScript (a variable only accessible through functions that close over it, with no other way to reach it directly) and memoization (caching an expensive computation's result inside a closure between calls). Each call to a factory function that returns a closure creates its own, completely independent set of captured variables.

```javascript
function createCounter() {
  let count = 0; // private — unreachable except through the returned functions
  return {
    increment: () => ++count,
    reset: () => { count = 0; },
    getValue: () => count,
  };
}

const counterA = createCounter();
const counterB = createCounter();
counterA.increment();
counterA.increment();
console.log(counterA.getValue(), counterB.getValue()); // 2 0 — fully independent
```

4. Understanding this: Four Binding Rules

this is determined by how a function is called, not where it's defined — with four main rules. Default binding: a plain function call (greet()) has this as undefined in strict mode (or the global object otherwise). Implicit binding: calling a function as a method (obj.greet()) sets this to obj. Explicit binding: call(), apply(), or bind() set this directly, overriding the other rules. New binding: calling a function with new creates a brand-new object and sets this to it. Arrow functions follow none of these — they capture this lexically from their enclosing scope at the moment they're defined, which is exactly why they're the standard choice for callbacks that need to keep referring to an outer this.

```javascript
const obj = {
  name: "Widget",
  logRegular: function () {
    setTimeout(function () {
      console.log(this?.name); // undefined — plain function loses `this`
    }, 0);
  },
  logArrow: function () {
    setTimeout(() => {
      console.log(this.name); // "Widget" — arrow inherits `this` from logArrow
    }, 0);
  },
};
```

5. call, apply, and bind

call(thisArg, arg1, arg2, ...) and apply(thisArg, argsArray) both invoke a function immediately with an explicitly chosen this, differing only in how they accept arguments — individually versus as an array. bind(thisArg, ...) does not invoke the function at all; instead it returns a brand-new function permanently bound to the given this (and optionally some leading arguments pre-filled), which is especially useful for passing an object's method as a callback elsewhere without losing its intended this.

```javascript
function introduce(greeting) {
  return `${greeting}, I'm ${this.name}`;
}

const person = { name: "Maya" };
console.log(introduce.call(person, "Hi"));   // "Hi, I'm Maya"
console.log(introduce.apply(person, ["Hey"])); // "Hey, I'm Maya"

const boundIntroduce = introduce.bind(person);
console.log(boundIntroduce("Hello")); // "Hello, I'm Maya" — `this` locked in permanently
```

6. Higher-Order Functions: map, filter, and reduce

A higher-order function either accepts a function as an argument, returns a function, or both — map(), filter(), and reduce() are the three most-used examples, all accepting a callback and all returning a new array (map/filter) or a single accumulated value (reduce) without mutating the original array. map() transforms every element 1-to-1; filter() keeps only elements the callback returns true for; reduce() folds the whole array down to one value by repeatedly combining an accumulator with each element, starting from a supplied initial value.

JAVASCRIPT

```
const orders = [
  { item: "Pen", qty: 3, price: 10 },
  { item: "Notebook", qty: 2, price: 50 },
  { item: "Eraser", qty: 5, price: 5 },
];

const totals = orders.map((o) => o.qty * o.price);        // [30, 100, 25]
const expensive = orders.filter((o) => o.price > 20);      // [{ item:
"Notebook", ... }]
const grandTotal = orders.reduce((sum, o) => sum + o.qty * o.price, 0); // 155
```

7. IIFEs and the Classic Module Pattern

An Immediately Invoked Function Expression — (function () { ... })() — defines and calls a function in one statement, creating a private scope that executes once and disappears, which was the standard pre-ES6 way to avoid leaking variables into the global scope when ES modules didn't yet exist. The 'module pattern' built on top of this: an IIFE that declares private variables internally and returns an object exposing only the methods meant to be public, directly mirroring what the factory-function closures shown earlier achieve — understanding this older pattern makes it clear that ES modules formalized a capability closures already provided, rather than inventing something entirely new.

JAVASCRIPT

```
const calculator = (function () {
  let total = 0; // private, inaccessible from outside the IIFE
  return {
    add: (n) => (total += n),
    getTotal: () => total,
  };
})(); // invoked immediately

calculator.add(5);
console.log(calculator.getTotal()); // 5
```

8. Currying and Partial Application

Currying transforms a function that takes multiple arguments into a sequence of functions that each take one argument, returning the next function in the chain until all arguments have been supplied — add(1)(2)(3) instead of add(1, 2, 3). Partial application is the related, slightly looser idea of pre-filling some of a function's arguments (often via bind()) to produce a more specific function for reuse later. Both techniques lean entirely on closures: each returned inner function closes over the arguments already supplied by the outer calls.

```javascript
const add = (a) => (b) => (c) => a + b + c;
console.log(add(1)(2)(3)); // 6

const addFive = add(5); // partially applied — `a` is locked in as 5
console.log(addFive(10)(20)); // 35
```

Common Pitfalls

- Using a regular function as an object method's callback and being surprised that `this` is no longer the object inside the callback.
- Expecting an arrow function used directly as an object's method to have `this` bound to that object — it inherits `this` from the surrounding scope instead.
- Assuming all calls to a closure-returning factory function share the same captured state — each call creates an entirely independent set of variables.
- Relying on the `arguments` object inside an arrow function — arrow functions have no `arguments` of their own; it resolves to the enclosing function's.
- Calling bind() repeatedly inside a loop or on every render instead of once, creating a new bound function reference each time unnecessarily.
- Calling reduce() on a possibly-empty array without supplying an initial value, causing a runtime TypeError.
- Confusing call/apply (invoke immediately) with bind (returns a new function to invoke later) and using the wrong one for the situation.

Key Takeaways

- Function declarations are fully hoisted; function expressions and arrow functions are not usable before their assignment line runs.
- Scope is lexical — determined by where code is written, not by which function calls which at runtime.
- A closure lets an inner function keep access to its outer function's variables long after the outer function has returned — this is the basis of private state and memoization.
- this is set by how a function is called (default, implicit, explicit, or new binding); arrow functions opt out entirely and inherit this lexically.
- call/apply invoke immediately with a chosen this; bind returns a new function permanently bound to a chosen this for later use.
- map/filter/reduce are the core higher-order array functions — map transforms, filter selects, reduce accumulates.

Learn More

MDN — Closures
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures

MDN — this
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/this

javascript.info — Function binding
https://javascript.info/bind